



NDN-DPDK File Server for Data-Intensive Science Applications

Junxiao Shi, Davide Pesavento, Lotfi Benmohamed

Communications Technology Laboratory

National Institute of Standards and Technology

Quick Recap from 2018-2020

- NDN-DPDK started as a high-speed NDN forwarder.
- 100 Gbps forwarding speed achieved in 2020. How did we get there?
- Adopt better algorithms and data structures.



- Efficient packet decoder.
- 2-stage LPM in FIB.
- PIT-CS Composite Table.

- Reduce overhead in library and kernel.



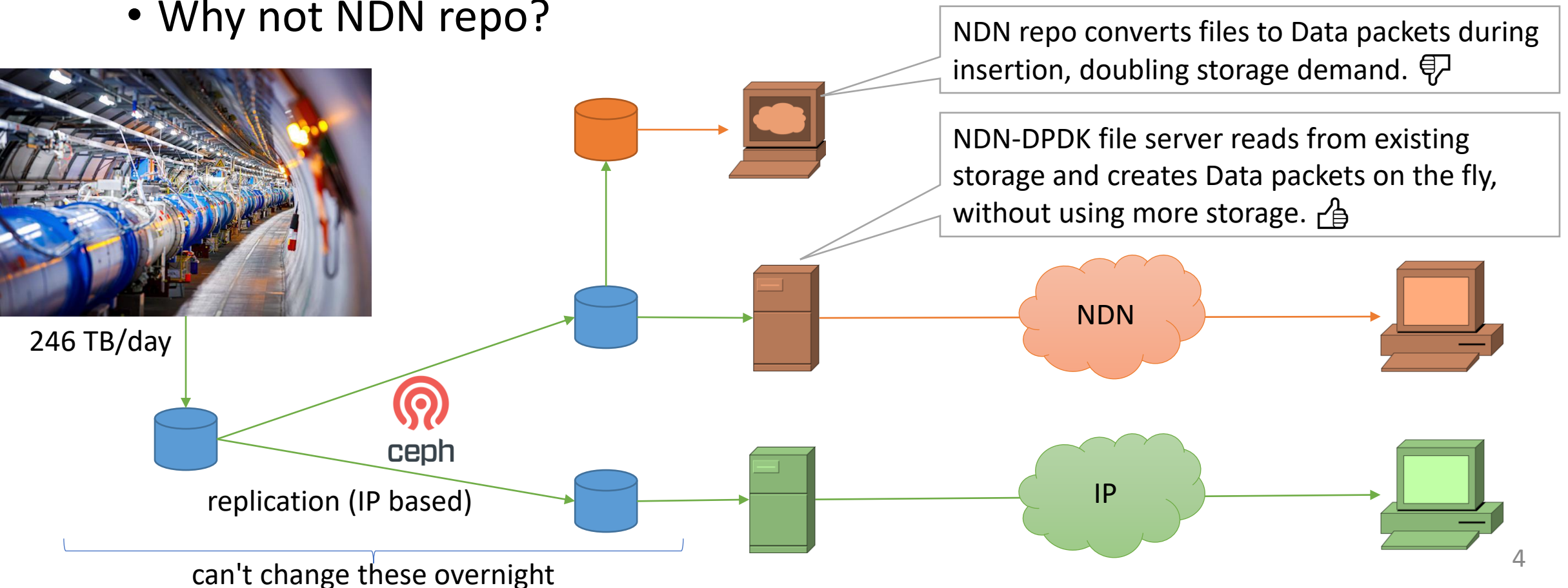
- Multi-threaded architecture.
- Hugepage memory pools.
- Hardware offloads.

File Server

ndncatchunks /files/code/ndn-dpdk.tgz

Motivation for a File Server

- High speed forwarder goes well with fast applications.
- Data-intensive science uses ~~big data~~ many large files.
- Why not NDN repo?



File Server Protocol

1. Retrieve metadata.

`/files/dir/file/32=metadata`

Metadata: result of STATX syscall.

- Versioned name (derived from modify time).
- File type and Unix permissions.
- File size, segment size, number of segments.
- Create time, modify time.

This format extends the RDR protocol.

File does not exist – application Nack.

2. Fetch file content.

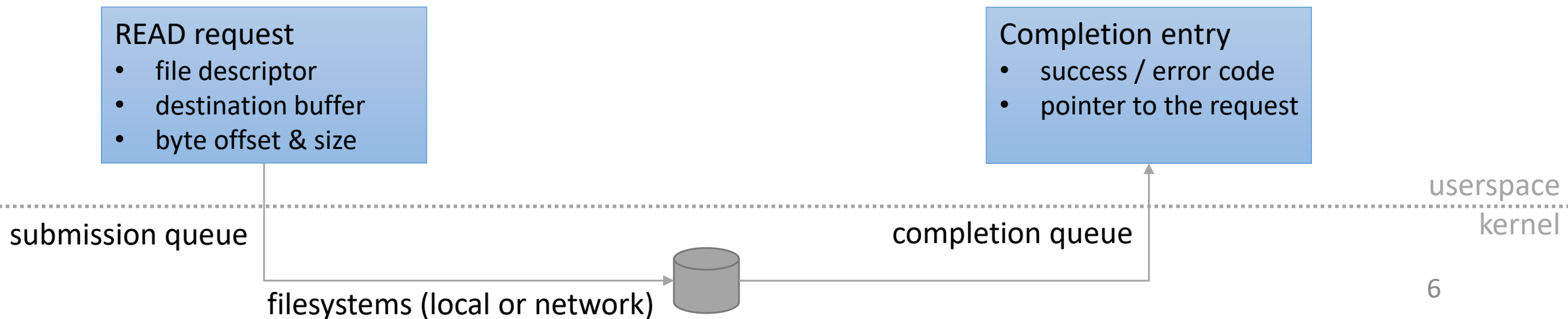
`/files/dir/file/v=7/seg=0`

Encryption and signing:

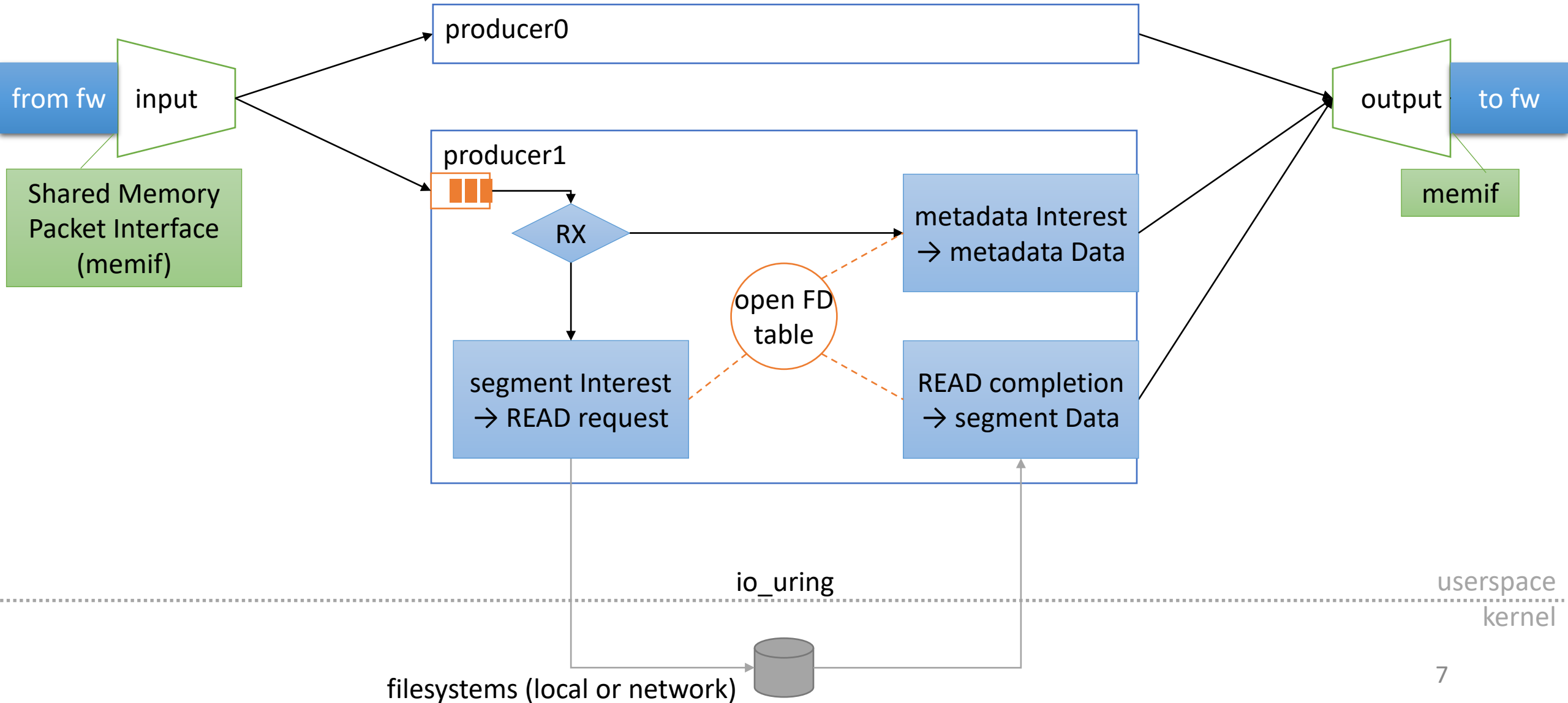
- Not needed in current use case.
- Possible with hardware crypto accelerator.

io_uring Introduction

- io_uring: new asynchronous I/O API for Linux.
 - Fewer syscalls than `read()` or `aio_read()`.
 - Available since Linux kernel 5.2, with enhancements in later versions.



File Server Architecture



Open File Descriptors Table

- Purpose: reuse a file descriptor for Interests accessing the same file.
- Hash table of file descriptors, indexed by file name.
- Cleanup queue for no-longer-in-use file descriptors.

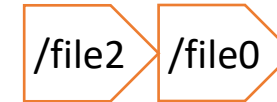
How many ongoing READ requests are using this file?

cached metadata

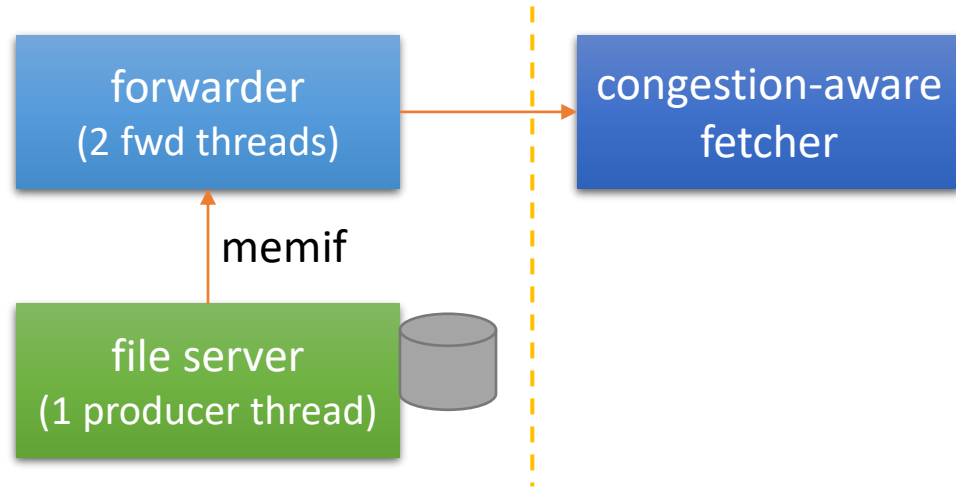
name	fd	refcount	metadata
/file0	1004	0	154 MiB, 08:17, ...
/file1	1003	2	283 MiB, 14:30, ...
/file2	1007	0	781 MiB, 17:01, ...

Only contains files not referenced by ongoing READ requests.
When full, `close()` oldest file.

cleanup queue
(limited capacity)



Benchmark Setup



Two physical machines:

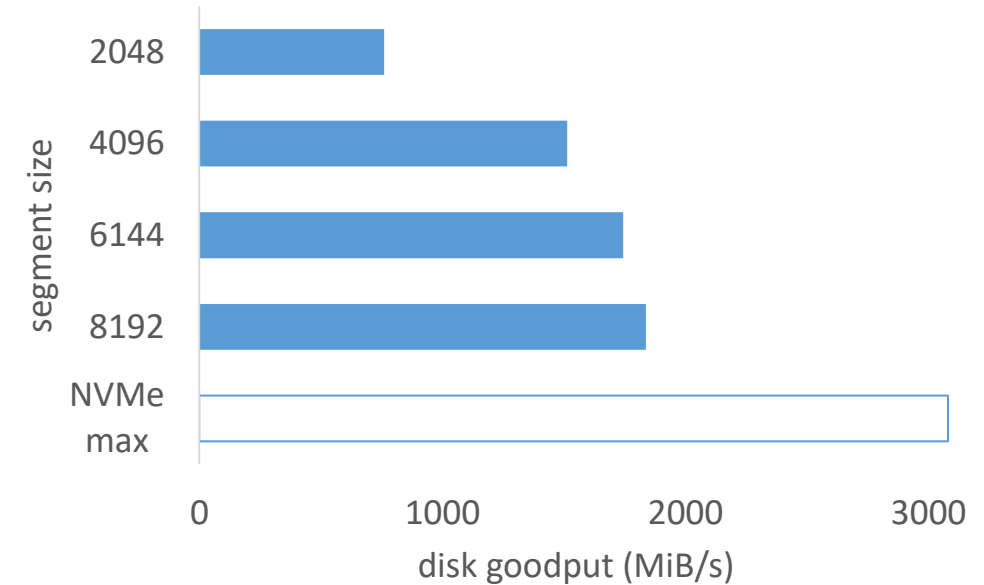
- file server + forwarder
- congestion-aware fetcher
 - CUBIC-like congestion control

- CPU: dual Intel Xeon Gold 6240
 - 2.60 GHz, Hyper Threading disabled
- Memory: 2933 MHz, four channels
 - 64x 1GB hugepages per NUMA socket
- NIC: Mellanox ConnectX-5 100 Gbps
 - MTU: 9000
- Storage: Intel 7600p NVMe
 - Sequential read: up to 3080 MiB/s
- Ubuntu 18.04, Linux kernel 5.4

Preliminary Benchmark Result

Task: transfer a single file

- File size: 20480 MiB
- not saving file to disk
- 5 trials, clear kernel page cache between trials



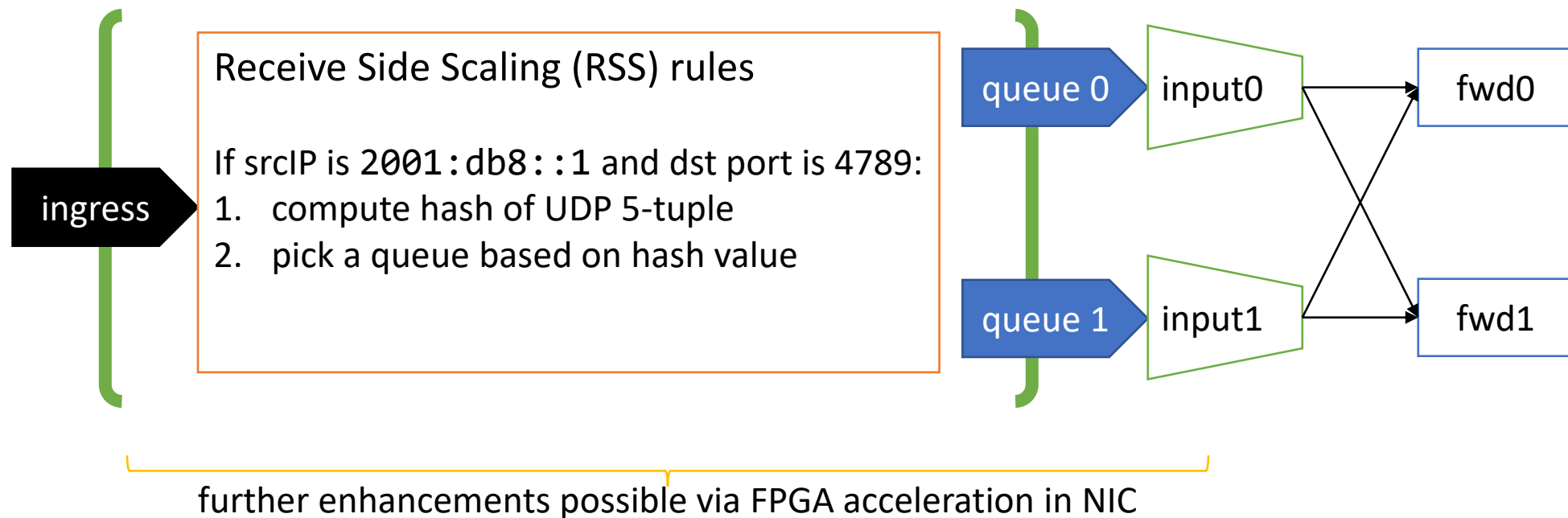
segment size	total segments	avg duration	network goodput	disk goodput	% NVMe max
2048 bytes	10,485,760	26.94 s	6.4 Gbps	760 MiB/s	25%
4096 bytes	5,242,880	13.54 s	12.7 Gbps	1,513 MiB/s	49%
6144 bytes	3,495,254	11.74 s	14.6 Gbps	1,744 MiB/s	57%
8192 bytes	2,621,440	11.14 s	15.4 Gbps	1,838 MiB/s	60%

Forwarder Updates

```
jq -n '{} ' | ndndpdk-ctrl activate-forwarder
```

Input Bottleneck & Receive Side Scaling

- Input thread was identified as a bottleneck of the forwarder.
- Forwarder can now use multiple input threads.
 - Subject to hardware/driver limitations (i.e., they only understand IP).
 - So far, only effective for VXLAN on Mellanox ConnectX-5.



Interactive Benchmark webapp

forwarder
(16 cores)

86:00:0
86:00:0
/A

traffic gen
(16 cores)

d8:00:0
d8:00:0
/B

face scheme

VXLAN

▼

face RX queues

2

Ethernet face can only use 1 RX queue

forwarding threads

6

payload length

8000

Interest name length

12

Data match

exact match

▼

trial duration

30

seconds

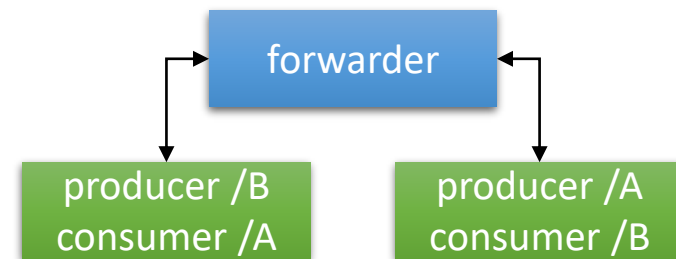
START

running trial 24

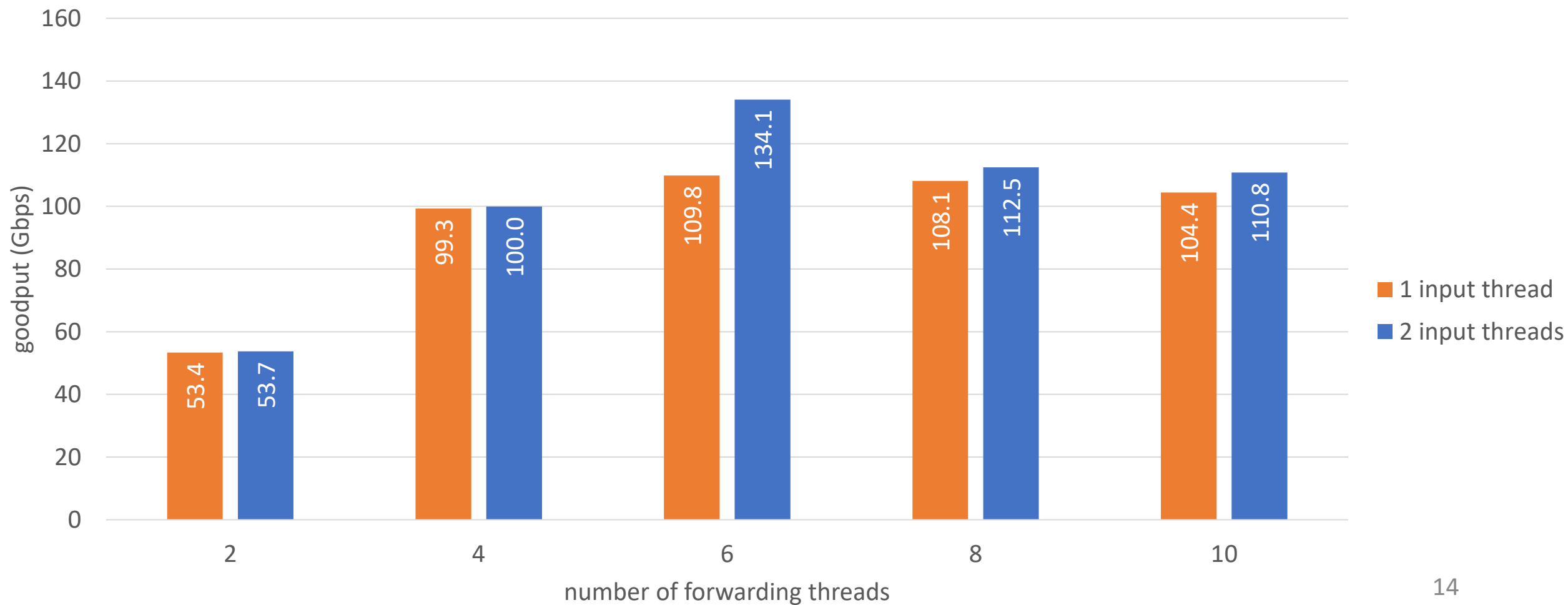
timestamp	Data packets throughput	goodput
10:59:34 AM	2.099 Mpps	134.353 Gbps
11:00:06 AM	2.097 Mpps	134.236 Gbps
11:00:38 AM	2.077 Mpps	132.947 Gbps
11:01:10 AM	2.097 Mpps	134.222 Gbps
11:01:43 AM	2.097 Mpps	134.191 Gbps
11:02:15 AM	2.084 Mpps	133.353 Gbps



Input Bottleneck Relieved

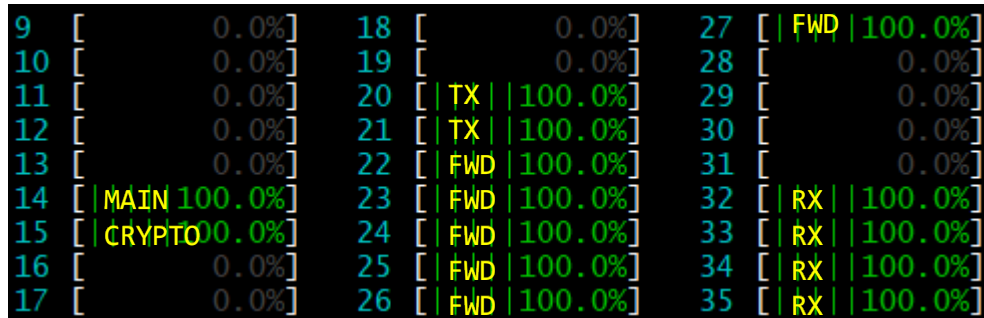


forwarder throughput with bidirectional traffic
(VXLAN, 12 name components, 8000-octet payload, same NUMA socket)

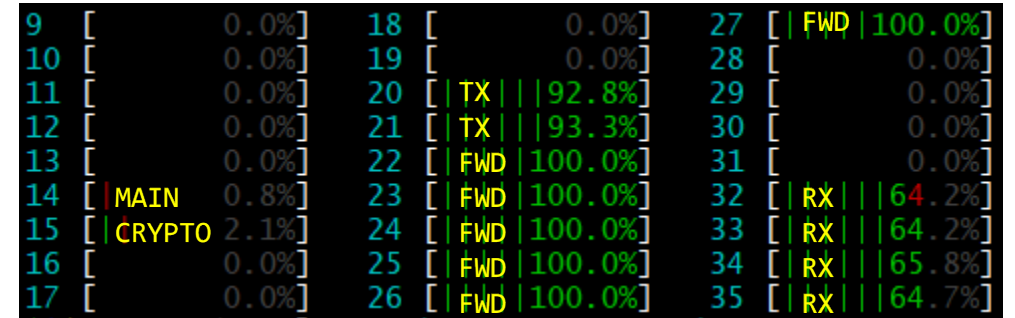


Idle CPU Usage Reduction

- YaNFD paper at ACM-ICN'21: "NDN-DPDK is using 100% CPU polling!"
 - Polling is fundamental to DPDK, but it doesn't have to use 100% CPU.
- THREADSLEEP compile time option:
 1. If a thread processed zero packets in a poll, it sleeps for 1ns.
 2. If idling continues, sleep duration is gradually increased up to 100us.



9 [0.0%]	18 [0.0%]	27 [FWD 100.0%]
10 [0.0%]	19 [0.0%]	28 [0.0%]
11 [0.0%]	20 [TX 100.0%]	29 [0.0%]
12 [0.0%]	21 [TX 100.0%]	30 [0.0%]
13 [0.0%]	22 [FWD 100.0%]	31 [0.0%]
14 [MAIN 100.0%]	23 [FWD 100.0%]	32 [RX 100.0%]
15 [CRYPTO 100.0%]	24 [FWD 100.0%]	33 [RX 100.0%]
16 [0.0%]	25 [FWD 100.0%]	34 [RX 100.0%]
17 [0.0%]	26 [FWD 100.0%]	35 [RX 100.0%]



9 [0.0%]	18 [0.0%]	27 [FWD 100.0%]
10 [0.0%]	19 [0.0%]	28 [0.0%]
11 [0.0%]	20 [TX 92.8%]	29 [0.0%]
12 [0.0%]	21 [TX 93.3%]	30 [0.0%]
13 [0.0%]	22 [FWD 100.0%]	31 [0.0%]
14 [MAIN 0.8%]	23 [FWD 100.0%]	32 [RX 64.2%]
15 [CRYPTO 2.1%]	24 [FWD 100.0%]	33 [RX 64.2%]
16 [0.0%]	25 [FWD 100.0%]	34 [RX 65.8%]
17 [0.0%]	26 [FWD 100.0%]	35 [RX 64.7%]

	100% polling	THREADSLEEP
throughput	134.1 Gbps	133.3 Gbps
CPU usage with traffic	1397%	1059%
CPU usage without traffic	1397%	55%

max = 1400%

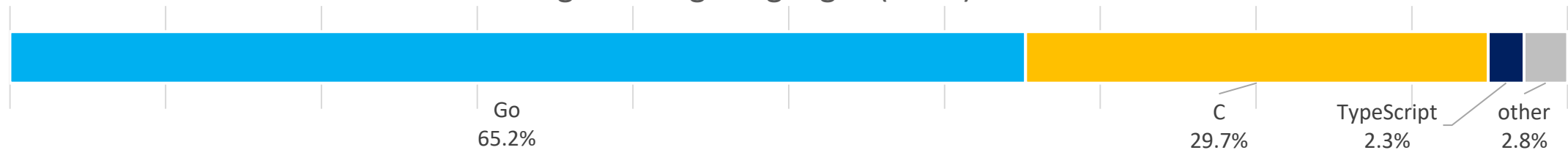
NDN-DPDK Codebase



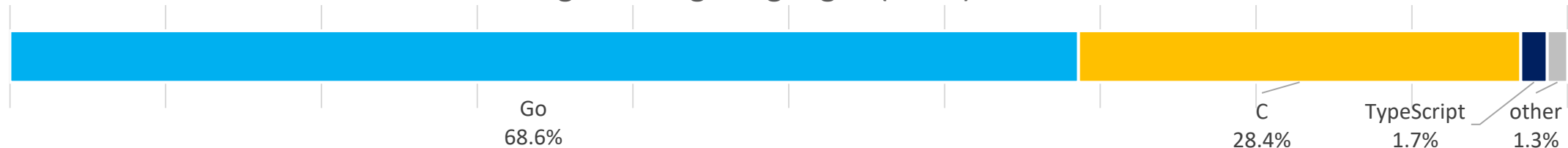
<https://github.com/usnistgov/ndn-dpdk>

- forwarder
- traffic generator
- file server
- user documentation
- GraphQL management tools
- interactive forwarder benchmark
- NDNgo library

Programming Languages (2020)



Programming Languages (2021)



Expanded Platform Support

new

- ✓ regularly tested
- known to work
- planned

- Operating systems:
 - ✓ Ubuntu 18.04 & 20.04
 - ✓ Debian 11
- Virtualization and containers:
 - ✓ Docker
 - KVM
 - VirtualBox
- Ethernet adapters:
 - ✓ Mellanox ConnectX-5
 - Intel i350 and Intel X710
 - other NICs via XDP driver (IP+NDN coexistence)
- Network transports:
 - ✓ Ethernet (+ VLAN)
 - UDP
 - ✓ VXLAN tunnel
- Local application support:
 - ✓ shared memory transport (memif)
 - ✓ NDNgo application library
- Experiment platforms:
 - ✓ Fed4FIRE & Emulab
 - Containernet emulator
 - FABRIC testbed

Q&A

Certain commercial entities, equipment, or materials may be identified in this document in order to describe an experimental procedure or concept adequately. Such identification is not intended to imply recommendation or endorsement by the National Institute of Standards and Technology, nor is it intended to imply that the entities, materials, or equipment are necessarily the best available for the purpose.